

Equipo formado por:*Moisés Martin Moreno**Alessia Papalia**Mario Inhiesto Sierra***Curso 2025-2026****1º DAM-DAW****REPOSITORIO DEL GITHUB****<https://github.com/Equipo-practica/Shop>**

El propósito de la App que creamos es para una pequeña tienda que posee una base de datos con clientes, pedidos y envíos.

Mediante el SceneBuilder preparamos todo el entorno visual, en IntelliJ la programación de la app, se utilizó MariaDB para el motor de la base de datos y almacenar/escribir la información.

Añadimos una ventana de “seguridad” para mantener nuestra “MainView” protegida, haciendo una vista login separada. Nuestra aplicación controller/Launcher.java (que es nuestro Main.java) carga la vista Login primero y si el login es correcto, carga el MainView. Solo se puede acceder al programa con un usuario (solo existe el usuario equipo) y una contraseña (que también es equipo).

También añadimos la validación en línea para que se le muestre al usuario mientras rellena el formulario. Para ello creamos en el package utils/Validador.java

En el modelo de datos añadimos una tercera entidad (Envíos) y se le implementa un CRUD completo para el mismo.

La aplicación fue traducida completamente al Inglés.

Añadimos un MenuBar en la parte superior (Edit) con la función Undo para Deshacer la última acción del usuario en el archivo GestionForm.fxml del paquete resources/vista y en el controlador GestionController, del paquete controller.

Y para el último apartado que realizamos de nuestra aplicación se encuentra el Exportar los datos en JSON, para el cual utilizamos un Prompt con la IA que fue el siguiente y la misma nos devolvió una solución con los requisitos que nosotros le solicitamos y que dejaremos toda la info en la Reflexión tecnológica.

REFLEXIÓN TECNOLÓGICA

Solicitamos ayuda a Chatgpt con la creación de la función para exportar los datos de nuestras tablas en el TableView.fxml a archivos JSON con los siguientes requisitos:

Moisés Martin Moreno
Alessia Papalia
Mario Inhiesto Sierra



1º DAM-DAW

El sistema debe manejar valores nulos (especialmente en fechas).

Paso 1: La clase de utilidad (`ExportadorJSON.java`) Esta clase se encarga de convertir tus listas de objetos a formato texto JSON manualmente. He incluido métodos para escapar comillas y manejar fechas nulas para evitar errores.

```
package utils;

import model.Cliente;
import model.Envio;
import model.Pedido;
import java.io.FileWriter;
import java.io.IOException;
import java.util.List;

public class ExportadorJSON {

    // Método genérico para guardar el String en disco
    private static boolean escribirArchivo(String ruta, String contenido) {
        try (FileWriter writer = new FileWriter(ruta)) {
            writer.write(contenido);
            return true;
        } catch (IOException e) {
            e.printStackTrace();
            return false;
        }
    }

    // Evita romper el JSON si el texto contiene comillas
    private static String escaparTexto(String texto) {
        if (texto == null) return "";
        return texto.replace("\"", "\\\"");
    }

    public static boolean exportarClientes(List<Cliente> clientes, String ruta) {
        StringBuilder json = new StringBuilder("[\n");
        for (int i = 0; i < clientes.size(); i++) {
            Cliente c = clientes.get(i);
            json.append(String.format("  {\n    \"id\": %d, \"nombre\": \"%s\", \"email\": \"%s\", \"telefono\": \"%s\", \"direccion\": \"%s\", \"fecha_registro\": \"%s\", \"fecha_ultima_actualizacion\": \"%s\", \"estado\": \"%s\", \"activo\": %s\n  }",
                c.getIdCliente(), escaparTexto(c.getNombre()), escaparTexto(c.getEmail()),
                c.getTelefono(), escaparTexto(c.getDireccion()), escaparTexto(c.getFechaRegistro()),
                escaparTexto(c.getFechaUltimaActualizacion()), escaparTexto(c.getEstado()),
                c.getActivo()));
            if (i < clientes.size() - 1) json.append(",");
            json.append("\n");
        }
        json.append("]");
        return escribirArchivo(ruta, json.toString());
    }
}
```

Equipo formado por:*Moisés Martín Moreno**Alessia Papalia**Mario Inhiesto Sierra***Curso 2025-2026****1º DAM-DAW**

Paso 2: Integración en el Controlador (TableController.java) En tu controlador, vinculamos la acción del menú con la lógica de exportación usando un selector de carpetas nativo.

```
@FXML
private void handleExportarJson(ActionEvent event) {
    // 1. Abrir selector de carpetas
    DirectoryChooser directoryChooser = new DirectoryChooser();
    directoryChooser.setTitle("Select folder to save JSONs");

    Stage stage = (Stage) tablaClientes.getScene().getWindow();
    File selectedDirectory = directoryChooser.showDialog(stage);

    if (selectedDirectory != null) {
        // 2. Definir rutas
        String rutaClientes = new File(selectedDirectory, "clientes.json").getAbsolutePath();

        // 3. Llamar a la utilidad pasando los items actuales de la tabla
        boolean exito = ExportadorJSON.exportarClientes(tablaClientes.getItems(), rutaClientes);

        if (exito) {
            mostrarAlerta(Alert.AlertType.INFORMATION, "Success", "Files exported successfully");
        } else {
            mostrarAlerta(Alert.AlertType.ERROR, "Error", "Could not save files.");
        }
    }
}
```

Y siguiendo los pasos modificamos nuestro controlador (TableController.java) vinculando al menú de acción un selector de carpetas nativo del sistema, que de ser “correcto” nos creara los .JSON de cada entidad en el directorio que seleccionemos.